

Architecting for Action:

Data Lineage and Analysis via the T-DAR Framework

Tim Didelot
contact@timdidelot.fr

September 2026

Abstract

Data lineage is a critical aspect of modern data management, providing insights into the flow and transformation of data across complex ecosystems. This paper introduces the T-DAR framework, a dynamic approach to analyzing data lineage that emphasizes the relationship between targeted observations, data assets, and actionable results. By leveraging the T-DAR framework, organizations can enhance their understanding of data processes, improve decision-making, and ensure compliance with regulatory requirements.

1 Introduction

In the era of Big Data, organizations have adopted a data-driven approach to decision-making. However, the complexity of data ecosystems often leads to challenges in understanding the flow and transformation of data [1]. This paper introduces the **T-DAR framework**, which stands for Targeted Data-Actions-Results, as a methodology for analyzing data lineage and ensuring actionable insights.

The T-DAR framework provides a structured approach to capturing and analyzing the relationships between data assets, the actions taken on them, and the resulting outcomes. By focusing on targeted observations, organizations can gain a deeper understanding of their data processes and make informed decisions based on actionable insights.

2 T-DAR Framework Overview

The T-DAR framework is inspired by the Data-Action-Response (DAR) model of most healthcare information systems [2].

2.1 Architecture

As illustrated in Figure 1, the architecture of the T-DAR framework is centered around a *Targeted Observation*, which is directly linked to a specific *Data Asset* within the ecosystem. Rather than treating data lineage as a static representation of data streams, the T-DAR framework takes advantage of the dynamic nature of data environments, allowing for real-time analysis and decision-making. It provides a way to analyse not only results, but also the medium through which the results are obtained, and the actions that lead to those results.

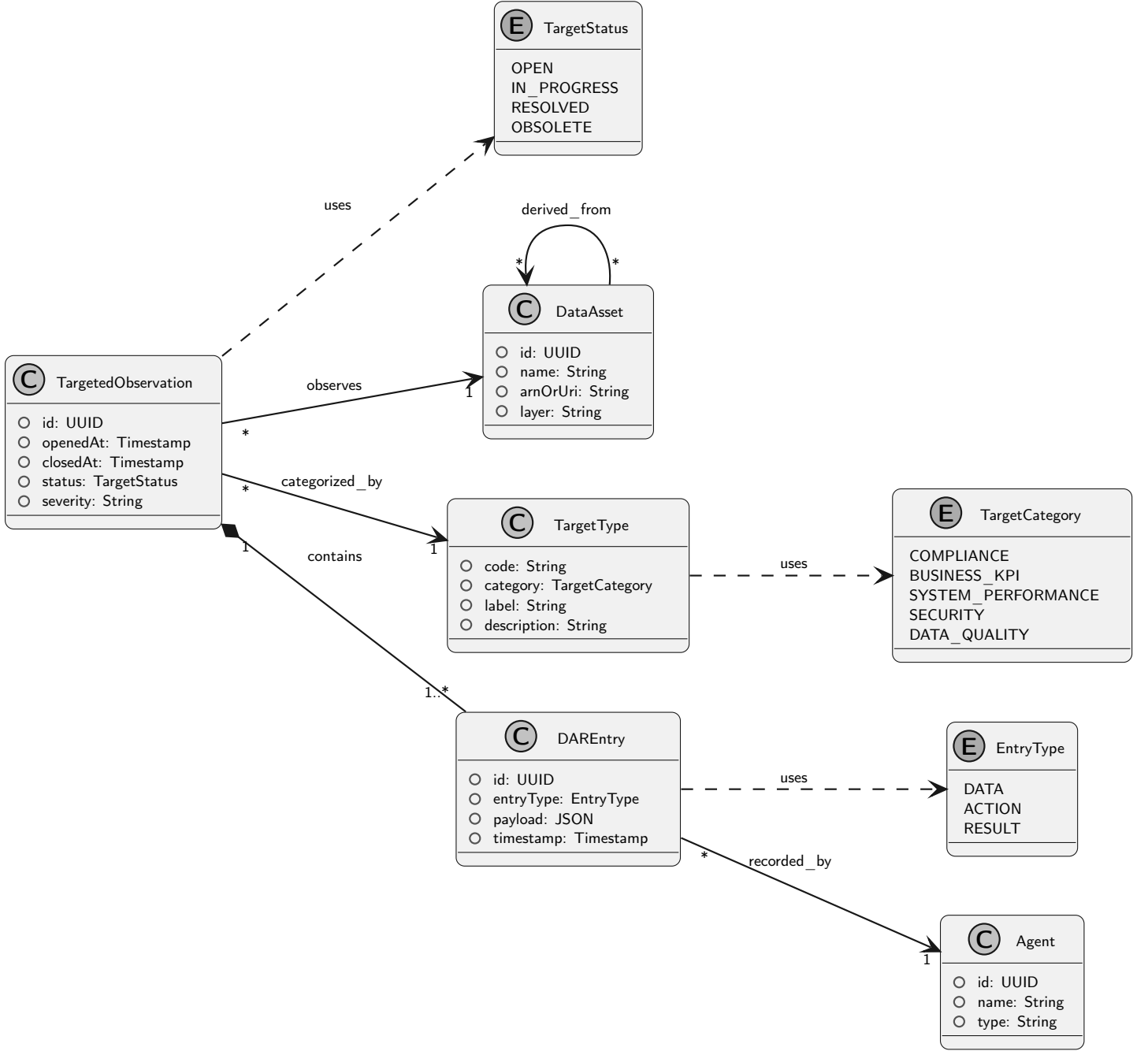


Figure 1: T-DAR Architecture Overview

To operationalize this dynamic analysis, every *Targeted Observation* acts as a container for a chronological sequence of *DAREntry* records. These entries are categorized into three distinct phases:

- **Data (D) – The Assessment:** A Data entry represents a factual snapshot, an audit trail, or a metric collected from the *Data Asset*.
- **Action (A) – The Intervention:** An Action entry represents a specific action taken in response to the data or the results of the data.
- **Result (R) – The Outcome:** A Result entry represents the outcome or consequence of the preceding action taken.

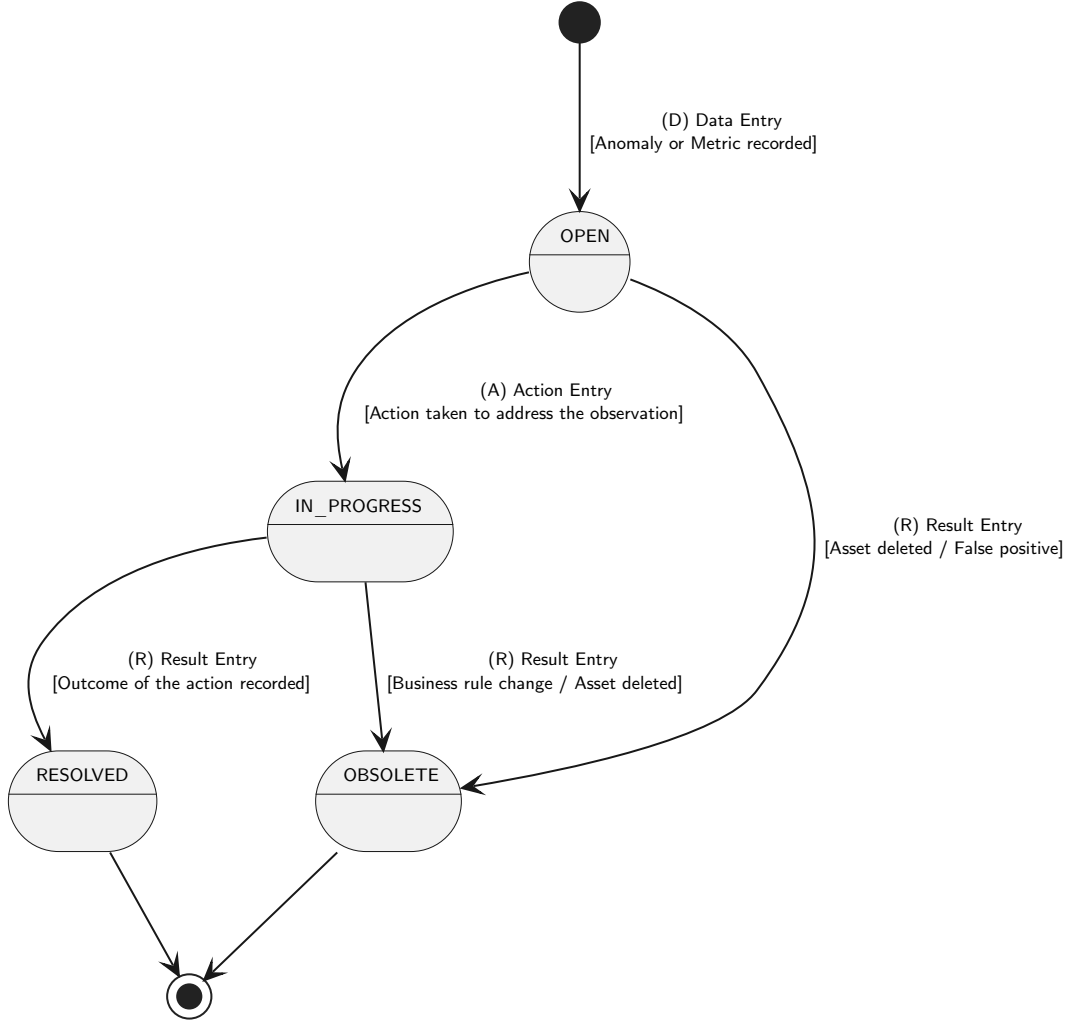


Figure 2: T-DAR State Diagram

As shown in Figure 2, these DAR entries act as triggers that drive the lifecycle of a *Targeted Observation*, transitioning it from an initial open state to a final resolution.

Furthermore, since data assets are inherently interconnected through a **derived_from** lineage relationship, the T-DAR framework allows organizations to track how an *Action* applied to an upstream asset propagates its *Results* to downstream dependencies. This elevates traditional data lineage from a simple dependency graph to an actionable governance tool.

2.2 Handling Fine-Grained Lineage

A critical strength of the T-DAR framework is its ability to manage complex, fine-grained lineage. While traditional models often stop at the table or system level, a *Data Asset* in T-DAR operates as a polymorphic entity. It can represent a macro-level database, a micro-level column, or even a processing function.

Consequently, an observation simply references a granular identifier, such as `urn:asset:silver:transactions:transaction_amount`, without altering its own generic data model. This precision ensures that downstream impact analysis is exact, allowing data teams to isolate and resolve granular issues without invalidating or locking entire tables.

2.3 Core Data Model

To establish rigorous guarantees on data consistency, T-DAR relies on a structured data model. The system operates over a dependency graph where each asset exposes its downstream impact.

Core Data Structures. The framework is built on two primary objects: the `DAREntry` and the `TargetedObservation`.

```
struct DAREntry {
    UUID id;
    EntryType entryType; // DATA, ACTION, or RESULT
    Timestamp timestamp; // Event execution time
    String agentId;       // Human or AI agent identifier
    JSON payload;         // Contextual data and metrics
}

class TargetedObservation {
    UUID id;
    String dataAssetId; // URI of the targeted data asset
    String targetType;  // e.g., DATA_QUALITY, SECURITY
    TargetStatus status; // OPEN, IN_PROGRESS, RESOLVED, OBSOLETE
    List<DAREntry> log;  // Chronological history of events
}
```

State Machine Lifecycle. The lifecycle of an observation is governed by a deterministic state machine:

$$\begin{aligned} \text{NULL} + \text{DATA} &\longrightarrow \text{OPEN} \\ \text{OPEN} + \text{ACTION} &\longrightarrow \text{IN_PROGRESS} \\ \text{OPEN} + \text{RESULT} &\longrightarrow \text{OBSOLETE} \\ \text{IN_PROGRESS} + \text{RESULT} &\longrightarrow \text{RESOLVED} \end{aligned}$$

Any transition not explicitly covered by this state machine throws an exception, guaranteeing that no corrective action can be recorded without prior anomaly detection.

3 Case Study: Applying T-DAR to a Medallion Architecture

To demonstrate practical value, we evaluate the framework within a standard medallion architecture pipeline [3], structured into Bronze (Raw), Silver (Cleaned), and Gold (Aggregated) layers.

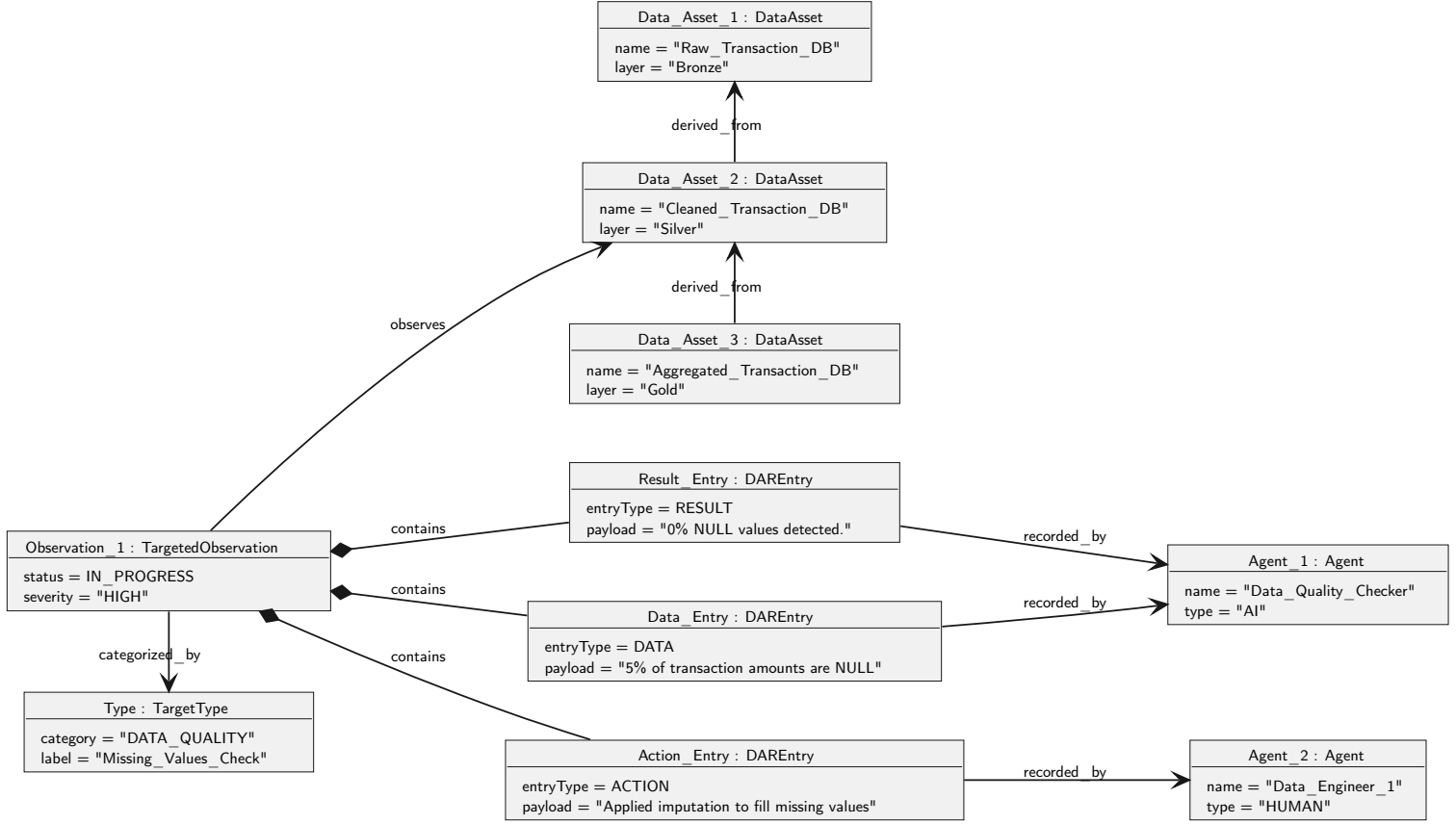


Figure 3: T-DAR Applied to a Medallion Architecture

As depicted in Figure 3, the framework provides end-to-end traceability when an anomaly occurs:

1. An automated agent (*Agent_1*) detects that 5% of values are missing in the Silver layer and initializes a *Targeted Observation* with a **DATA** entry.
2. Upstream-downstream lineage identification instantly warns stewards of propagation risks toward the Gold layer.
3. A data engineer (*Agent_2*) executes an imputation pipeline, appended as an **ACTION** entry.
4. Automated validation runs and records a **RESULT** entry confirming 0% missing values, marking the observation as **RESOLVED**.

3.1 Implementation Example: T-DAR API Payload

Listing 4 illustrates the JSON payload dispatched by the automated monitoring agent to register the incident.

```

{
  "observation_id": "obs-1",
  "target_type": "DATA_QUALITY",
  "data_asset_id": "urn:silver:tx:amount",
  "status": "OPEN",
  "dar_entry": {
    "entry_type": "DATA",
    "agent_id": "ai-checker-bot",
    "timestamp": "2026-09-22T10:00:00+02:00",
    "payload": {
      "metric": "null_values_percentage",
      "value": 0.05,
      "message": "5% of transaction amounts are NULL",
      "pipeline_run_id": "run-202609221000-0001"
    }
  }
}

```

Figure 4: Payload registering an initial Data Entry.

4 Related Work

Data lineage specifications such as OpenLineage [4] standardize passive metadata collection across pipeline runs. OpenLineage focuses on capturing job executions, inputs, and outputs to maintain dependency graphs. Observability tools build on this by alerting on data degradation.

However, existing frameworks primarily address anomaly detection (the *Data* phase) without tracking downstream interventions (*Action* and *Result*). T-DAR is designed to complement existing platforms: by running on top of OpenLineage dependency graphs, it connects operational alerts directly to remediation histories.

5 Conclusion and Broader Applications

This paper introduced the T-DAR framework, a structured architectural pattern that bridges the gap between static data lineage and active governance. By encapsulating issues within Targeted Observations and tracing the lifecycle through Data, Action, and Result (D-A-R) entries, organizations gain an immutable and actionable audit trail of their ecosystems.

While the primary case study focused on data quality in a medallion architecture, the T-DAR framework is fundamentally domain-agnostic:

- **Marketing and Campaign Tracking:** Documenting marketing initiatives where observations target specific demographics, trace campaign execution, and measure conversion ROI.
- **Hybrid Automated Controls:** Facilitating AI-driven anomaly detection combined with explicit human engineer approval workflows.
- **Universal Process Security:** Securing and auditing operational processes to ensure total accountability regarding who performed an intervention, why it was triggered, and what the final outcome was.

License

This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

References

- [1] S. Syed and R. C. R. Nampalli, “Data Lineage Strategies – A Modernized View,” *Educational Administration: Theory and Practice*, vol. 26, no. 4, pp. 965–973, 2020.

- [2] Lippincott Williams & Wilkins, *Portable RN: The All-in-One Nursing Reference*, 3rd ed., Lippincott Williams & Wilkins, 2007, p. 678.
- [3] Databricks, “What is a Medallion Architecture?,” *Databricks Glossary*, 2026.
- [4] LF AI & Data Foundation, “OpenLineage: An Open Standard for Data Lineage,” <https://openlineage.io>, 2026.